

Détection de Fraude par Carte Bancaire : Modélisation par Régression Logistique et Optimisation du Risque

Yves GNONHOUE

yves.gnonhoue@etu.univ-angers.fr

Année académique 2025-2026

1 Introduction

Aujourd'hui, avec la digitalisation des services bancaires, on constate une profonde modification des habitudes de paiement ainsi qu'une multiplication du volume des transactions financières. Cette évolution, portée par les avancées technologiques, favorise également le développement de fraudes bancaires commises par des individus malveillants, ce qui est devenu un enjeu majeur. C'est dans ce contexte que, pour essayer d'éradiquer ces fraudes, les méthodes statistiques et les techniques d'apprentissage automatique permettent d'identifier rapidement les transactions suspectes.

Pour mettre en pratique ces méthodes et techniques d'apprentissage automatiques, nous avons décidé d'utiliser le jeu de donnée **creditcard**. Ce jeu de donnée contient les transactions par carte bancaire effectuées en septembre 2013 par des titulaires européens. Il présente les transactions réalisées sur deux jours et recense 492 fraudes sur 284 807 transactions. L'ensemble de données est fortement déséquilibré : les transactions frauduleuses ne représentent que 0,172 % du total.

2 Présentation du jeu de données

Le jeu de données **creditcard** est composé principalement de variables numériques issues d'une Analyse en Composantes Principales (ACP). Pour des raisons de confidentialité, les variables d'origine ne sont pas disponibles ; seules les variables **V1 à V28** correspondent aux composantes principales, tandis que les variables **Temps** (temps écoulé en secondes depuis la première transaction) et **Montant** (montant de la transaction) n'ont pas été transformées. Enfin, la variable **Classe** constitue la variable cible, prenant la valeur 1 pour une transaction frauduleuse et 0 sinon ; en raison du fort déséquilibre entre les classes, l'évaluation du modèle repose de préférence sur l'aire sous la courbe précision-rappel (AUPRC), plus adaptée que la simple précision issue de la matrice de confusion.

3 Description et exploration des données

Avant de procéder à l'estimation de modèles de régression, il est fondamental de mener une analyse descriptive exploratoire. Cette étape permet de valider la cohérence de notre sous-échantillon d'étude et de mettre en lumière les premières tendances univariées ainsi que la dispersion de nos variables.

3.1 Aperçu des dix premières observations

TABLE 1 – Aperçu des premières lignes du jeu de données (Structure brute)

Id	Time	V1	V2	V3	V4	...	V28	Amount	Class (Y)
0	0,0	-1,3598	-0,0728	2,5363	1,3782	...	-0,0211	149,62	0
1	0,0	1,1919	0,2662	0,1665	0,4482	...	0,0147	2,69	0
2	1,0	-1,3584	-1,3402	1,7732	0,3798	...	-0,0598	378,66	0
3	1,0	-0,9663	-0,1852	1,7930	-0,8633	...	0,0615	123,50	0
4	2,0	-1,1582	0,8777	1,5487	0,4030	...	0,2152	69,99	0
5	2,0	-0,4260	0,9605	1,1411	-0,1683	...	0,0811	3,67	0
6	4,0	1,2297	0,1410	0,0454	1,2026	...	0,0052	4,99	0
7	7,0	-0,6443	1,4180	1,0744	-0,4922	...	-1,0853	40,80	0
8	7,0	-0,8943	0,2862	-0,1132	-0,2715	...	0,1424	93,20	0
9	9,0	-0,3383	1,1196	1,0444	-0,2222	...	0,0831	3,68	0

Le tableau 1 présente les données réelles pour les dix premiers départements du sous-échantillon d'étude.

3.2 Analyse descriptive du jeu de données

TABLE 2 – Statistiques descriptives du jeu de données Credit Card Fraud Detection

Métrique	Time	V1	V2	...	Amount	Class (Y)
count	284 807, 0	$2,848 \times 10^5$	$2,848 \times 10^5$	...	284 807, 0	284 807, 0
mean	94 813, 9	$1,175 \times 10^{-15}$	$3,385 \times 10^{-16}$	...	88, 35	0, 0017
std	47 488, 1	1, 959	1, 651	...	250, 12	0, 0415
min	0, 0	-56, 407	-72, 716	...	0, 0	0, 0
25%	54 201, 5	-0, 920	-0, 599	...	5, 60	0, 0
50%	84 692, 0	0, 018	0, 065	...	22, 00	0, 0
75%	139 320, 5	1, 316	0, 804	...	77, 17	0, 0
max	172 792, 0	2, 455	22, 058	...	25 691, 16	1, 0

Le tableau 2 présente l'analyse descriptive des variables numériques met en évidence plusieurs caractéristiques importantes. Tout d'abord, aucune valeur manquante n'a été détectée dans le jeu de données, ce qui facilite les étapes de préparation avant l'apprentissage du modèle. Ensuite, la variable **Montant** présente une forte dispersion : la moyenne des transactions est de 88,35 €, tandis que la médiane est de 22 €, indiquant une distribution fortement asymétrique avec quelques transactions de montant élevé. La valeur maximale observée est de 25 691,16 €, qui correspond à une transaction exceptionnelle mais conservée dans l'analyse, car elle peut représenter un comportement réel dans un contexte bancaire.

Enfin, l'analyse de la variable cible montre un fort déséquilibre entre les classes. La proportion moyenne de la classe frauduleuse est de 0,001727, soit environ 0,1727 % des transactions. Cette caractéristique doit être prise en compte lors de l'évaluation du modèle, car une simple mesure de précision pourrait être trompeuse dans un problème où les transactions normales sont largement majoritaires.

4 Modélisation statistique

Dans cette section, nous présentons le cadre statistique de la régression logistique, appartenant à la famille des Modèles Linéaires Généralisés noté **GLM**, appliquée à la classification des transactions bancaires.

4.1 Prétraitement et standardisation des variables

Soit un échantillon de n transactions indépendantes $(Y_i, \mathbf{X}_i)_{1 \leq i \leq n}$. La variable cible binaire $Y_i \in \{0, 1\}$ correspond à la colonne **Class** (1 pour une fraude, 0 pour une transaction légitime).

Afin d'éviter que les différences d'échelle ne biaisent l'estimation des coefficients ou la convergence des algorithmes d'optimisation, les variables **Time** et **Amount** ont été centrées et réduites à l'aide du module `StandardScaler` de `scikit-learn` :

$$\tilde{X}_{ij} = \frac{X_{ij} - \bar{X}_j}{\sigma_j} \quad (1)$$

L'ensemble des $p = 30$ covariables explicatives est noté $\mathbf{X}_i = (1, X_{i1}, \dots, X_{i30})^\top$, incluant la constante (intercept).

Nous rappelons que les autres variables sont issues d'une ACP afin de préserver la confidentialité des données. Ces composantes étant déjà centrées et réduites par construction, il n'était pas nécessaire de les normaliser à nouveau.

4.2 Formalisation du Modèle Logistique (GLM Binomial)

Conditionnellement aux caractéristiques $\mathbf{X}_i$, la variable aléatoire Y_i suit une loi de Bernoulli de paramètre $p(\mathbf{X}_i) = \mathbb{P}(Y_i = 1 \mid \mathbf{X}_i)$, représentant la probabilité de fraude :

$$Y_i \mid \mathbf{X}_i \sim \mathcal{B}(p(\mathbf{X}_i)) \quad (2)$$

La régression logistique modélise le logarithme du rapport des chances (*log-odds*) par une combinaison linéaire des régresseurs :

$$\text{logit}(p(\mathbf{X}_i)) = \ln \left(\frac{p(\mathbf{X}_i)}{1 - p(\mathbf{X}_i)} \right) = \beta_0 + \sum_{j=1}^{30} \beta_j X_{ij} = \mathbf{X}_i^\top \boldsymbol{\beta} \quad (3)$$

Par inversion de la fonction de lien logit, la probabilité d'occurrence de la fraude s'exprime par la fonction sigmoïde :

$$p(\mathbf{X}_i) = \mathbb{P}(Y_i = 1 \mid \mathbf{X}_i) = \frac{1}{1 + e^{-\mathbf{X}_i^\top \boldsymbol{\beta}}} \quad (4)$$

où $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_{30})^\top$ est le vecteur des coefficients à estimer.

4.3 Estimation des paramètres et Inférence

Les coefficients $\boldsymbol{\beta}$ sont estimés par la méthode du **Maximum de Vraisemblance** (MV). La log-vraisemblance $\ell(\boldsymbol{\beta})$ s'écrit :

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n \left[y_i \mathbf{X}_i^\top \boldsymbol{\beta} - \ln \left(1 + e^{\mathbf{X}_i^\top \boldsymbol{\beta}} \right) \right] \quad (5)$$

L'optimisation numérique est réalisée via l'algorithme itératif de Newton-Raphson.

Pour évaluer l'impact individuel de chaque variable, nous réalisons le **test de Wald** sous l'hypothèse nulle $H_0 : \beta_j = 0$. La statistique de test s'écrit :

$$W_j = \frac{\hat{\beta}_j}{\widehat{\text{SE}}(\hat{\beta}_j)} \sim \mathcal{N}(0, 1) \quad (6)$$

Les variables présentant une p -value supérieure au seuil de significativité $\alpha = 0,05$ sont éliminées pour conserver un modèle parcimonieux et facilement interprétable sur le plan métier.

4.4 Modèle Initial et Sélection de Variables

Dans cette section, l'objectif est d'ajuster un modèle de régression logistique initial contenant l'ensemble des variables explicatives disponibles. Ce modèle de départ sert de point de référence pour identifier quelles variables contribuent réellement à la prédiction et quelles variables n'apportent que du bruit statistique. Nous appliquons ensuite une méthode d'élimination descendante (*backward elimination*) afin de simplifier le modèle tout en préservant (ou en améliorant) sa qualité globale.

Pour décider du retrait d'une variable à chaque étape, nous nous appuyons sur le test de Wald (représenté par la statistique z et sa p -value associée $P > |z|$) :

- Test de Wald (p -value) : Il teste l'hypothèse nulle (H_0) selon laquelle le coefficient de la variable est nul ($\beta_i = 0$). Une p -value supérieure au seuil usuel de 0,05 indique que la variable n'a pas d'effet statistiquement significatif sur la variable réponse.
- Pourquoi éliminer les variables inutiles ? Conserver des variables non significatives augmente la complexité du modèle sans améliorer sa précision, ce qui peut provoquer du surapprentissage (*overfitting*) et dégrader la précision des estimations des autres coefficients. On retire donc les variables une à une, en commençant par celle qui a la p -value la plus élevée.
- Critère d'Information d'Akaike (AIC) : À chaque suppression, nous vérifions que le score AIC diminue. Un AIC plus faible confirme que le modèle simplifié gagne en efficacité et en sobriété.

Résultats du Modèle Initial noté Modèle 1 (Étape 1)

Le tableau ci-dessous résume les estimations pour le modèle complet ($AIC = 2291,50$).

TABLE 3 – Estimations du modèle de régression logistique initial

Variable	Coefficient	Erreur Type	Statistique z	$P > z $
Intercept	−8,6656	0,147	−58,992	0,000
Time	−0,1777	0,107	−1,659	0,097
V1	0,0960	0,042	2,264	0,024
V2	0,0094	0,058	0,161	0,872
V3	−0,0079	0,053	−0,149	0,881
V4	0,6986	0,074	9,454	0,000
V5	0,1295	0,067	1,944	0,052
V6	−0,1198	0,074	−1,626	0,104
V7	−0,0969	0,067	−1,453	0,146
V8	−0,1739	0,030	−5,711	0,000
V9	−0,2843	0,111	−2,561	0,010
V10	−0,8176	0,097	−8,432	0,000
V11	−0,0621	0,081	−0,762	0,446
V12	0,0909	0,087	1,045	0,296
V13	−0,3312	0,082	−4,058	0,000
V14	−0,5571	0,062	−8,949	0,000
V15	−0,1141	0,086	−1,330	0,183
V16	−0,1908	0,125	−1,526	0,127
V17	−0,0216	0,070	−0,309	0,757
V18	−0,0131	0,129	−0,102	0,919 (→ 1 ^{ère} éliminée)
V19	0,0963	0,097	0,993	0,321
V20	−0,4582	0,082	−5,607	0,000
V21	0,3898	0,060	6,494	0,000
V22	0,6297	0,134	4,707	0,000
V23	−0,0951	0,058	−1,629	0,103
V24	0,1289	0,147	0,874	0,382
V25	−0,0761	0,131	−0,582	0,560
V26	0,0195	0,190	0,103	0,918
V27	−0,8188	0,122	−6,686	0,000
V28	−0,2937	0,088	−3,332	0,001
Amount	0,2291	0,094	2,449	0,014

AIC : 2291.5

Interprétation : La variable **V18** présente la p -value la plus élevée (0,919), bien au-dessus du seuil de 0,05. Elle sera donc supprimée en premier lors de la première étape d'élimination.

4.5 Modèle 2 (Étape 1)

Nous réajustons ensuite le modèle de régression logistique sur les variables restantes pour observer l'impact de cette suppression sur la qualité globale de nos prédictions.

TABLE 4 – Estimations du modèle de régression logistique après suppression de V_{18} (AIC = 2289,54)

Variable	Coefficient	Erreur Type	Statistique z	$P > z $
Intercept	-8,6651	0,147	-59,059	0,000
Time	-0,1789	0,106	-1,681	0,093
V1	0,0959	0,042	2,263	0,024
V2	0,0086	0,057	0,149	0,881
V3	-0,0081	0,053	-0,153	0,878
V4	0,6981	0,074	9,464	0,000
V5	0,1288	0,066	1,948	0,051
V6	-0,1192	0,073	-1,624	0,104
V7	-0,0972	0,067	-1,459	0,145
V8	-0,1738	0,030	-5,711	0,000
V9	-0,2853	0,110	-2,582	0,010
V10	-0,8167	0,096	-8,465	0,000
V11	-0,0635	0,080	-0,792	0,428
V12	0,0917	0,087	1,056	0,291
V13	-0,3306	0,081	-4,063	0,000
V14	-0,5566	0,062	-8,969	0,000
V15	-0,1137	0,086	-1,327	0,185
V16	-0,1985	0,099	-1,997	0,046
V17	-0,0241	0,066	-0,368	0,713
V19	0,0923	0,089	1,041	0,298
V20	-0,4600	0,080	-5,784	0,000
V21	0,3885	0,059	6,619	0,000
V22	0,6279	0,133	4,732	0,000
V23	-0,0953	0,058	-1,633	0,102
V24	0,1292	0,147	0,877	0,381
V25	-0,0767	0,131	-0,588	0,557
V26	0,0223	0,188	0,119	0,905
V27	-0,8186	0,122	-6,686	0,000
V28	-0,2940	0,088	-3,337	0,001
Amount	0,2283	0,093	2,451	0,014

AIC : 2289.54

Interprétation : Le Modèle 2 est meilleur que le Modèle 1 car son score AIC est plus bas (2289.54 contre 2291.50). Cette diminution confirme que le fait de retirer la variable V18 a rendu le modèle plus performant et plus simple, sans perdre d'information importante.

Conformément à notre méthode, nous continuons le processus pas à pas : à chaque étape, nous supprimons la variable ayant la p -value la plus élevée (la moins significative) afin d'obtenir à chaque fois le modèle ayant le meilleur AIC possible (le plus bas). La variable V26 présentant désormais la p -value la plus forte (0.905), elle est éliminée pour passer au Modèle 3.

4.6 Synthèse de la sélection de variables (Du Modèle 3 au Modèle 13)

Conformément à la méthode d'élimination descendante (*backward elimination*), nous avons poursuivi le retrait successif des variables non significatives. À chaque étape, la variable présentant la p -value la plus élevée a été éliminée afin de simplifier le modèle et d'optimiser le critère d'information d'Akaike (AIC).

Évolution du processus :

- **Étapes 3 à 10** : La suppression progressive des variables fortement non significatives (notamment V_3 , V_2 , V_{26} , V_{25} , V_{17} , etc.) a entraîné une baisse constante et continue de l'AIC, passant de 2287,56 à environ 2276,60.
- **Étapes 11 et 12 (Stagnation de l'AIC)** : Entre le 11^e et le 12^e modèle, le score AIC = 2275,91 est resté quasiment identique.
- **Étape 13** : La poursuite de l'élimination des dernières variables marginales (Time) non significatives ($p > 0,05$) a relancé la baisse de l'AIC pour atteindre environ 2275,71.

Bilan intermédiaire : Ce nettoyage progressif a permis de purger le modèle initial de son bruit statistique. Nous sommes passés de 30 variables explicatives à un sous-ensemble restreint, rendant le modèle plus sobre et plus robuste avant l'obtention du modèle final.

4.7 Étape 14 : Modèle final avec variables toutes significatives

Le tableau 5 présente les estimations du 14^e modèle (AIC = 2276,42) après élimination de la variable Time

TABLE 5 – Estimations du modèle de régression logistique épuré (Modèle 14)

Variable	Coefficient	Erreur Type	Statistique z	$P > z $
Intercept	-8,5873	0,131	-65,386	0,000
V1	0,0786	0,036	2,197	0,028
V4	0,6800	0,061	11,141	0,000
V5	0,1410	0,031	4,534	0,000
V7	-0,1183	0,056	-2,126	0,033
V8	-0,1489	0,023	-6,590	0,000
V9	-0,2229	0,083	-2,682	0,007
V10	-0,7746	0,086	-8,983	0,000
V13	-0,3031	0,079	-3,860	0,000
V14	-0,5128	0,051	-10,010	0,000
V16	-0,2351	0,060	-3,908	0,000
V20	-0,4104	0,076	-5,419	0,000
V21	0,3615	0,054	6,747	0,000
V22	0,5685	0,123	4,612	0,000
V23	-0,1057	0,052	-2,050	0,040
V27	-0,7429	0,122	-6,067	0,000
V28	-0,2780	0,091	-3,060	0,002
Amount	0,1897	0,072	2,644	0,008

Interprétation : À cette 14^e étape, nous constatons une légère augmentation du score AIC (2276,42). Cela indique que la dernière variable supprimée apportait une part d'information explicative dont la perte surpasse la réduction de complexité. Néanmoins, ce Modèle 14 présente l'avantage majeur d'avoir toutes ses variables explicatives significatives au seuil de 5% ($p < 0,05$).

Conformément au critère d'information d'Akaike (AIC), le Modèle 13 est retenu comme notre modèle final. Bien que le Modèle 14 ne conserve que des variables à p -value < 0.05 , la remontée du score AIC à l'étape 14 démontre que la suppression de la dernière variable a entraîné une perte

d'information prédictive. Le Modèle 13 offre donc le compromis optimal entre sobriété du modèle et qualité d'ajustement aux données.

5 Interprétation du modèle

Dans cette section, nous interprétons les résultats du modèle de régression logistique retenu (Modèle 13). Pour rendre ces résultats facilement compréhensibles, nous transformons les coefficients bruts (β_i) en Odds Ratios ($OR = e^{\beta_i}$), qui expriment la variation du risque relatif de fraude.

TABLE 6 – Estimations du modèle de régression logistique retenu (Modèle 13 – AIC = 2275,71)

Variable	Coefficient	Erreur Type	Statistique z	$P > z $
Intercept	−8,5902	0,132	−65,244	0,000
V1	0,0894	0,036	2,455	0,014
V4	0,6874	0,062	11,123	0,000
V5	0,1037	0,039	2,693	0,007
V6	−0,1033	0,065	−1,596	0,111
V7	−0,1040	0,056	−1,855	0,064
V8	−0,1757	0,029	−6,135	0,000
V9	−0,2360	0,084	−2,812	0,005
V10	−0,7562	0,087	−8,699	0,000
V13	−0,3034	0,078	−3,878	0,000
V14	−0,5014	0,052	−9,670	0,000
V16	−0,2372	0,060	−3,925	0,000
V20	−0,4208	0,075	−5,587	0,000
V21	0,3628	0,054	6,772	0,000
V22	0,5859	0,124	4,722	0,000
V23	−0,1020	0,050	−2,034	0,042
V27	−0,7455	0,123	−6,082	0,000
V28	−0,2816	0,089	−3,177	0,001
Amount	0,1971	0,071	2,776	0,006

5.1 Analyse des coefficients et des Odds Ratios

Rappelons qu'en régression logistique, nous avons les conditions suivantes :

- Un coefficient positif ($\beta > 0 \Rightarrow OR > 1$) indique que lorsque la variable augmente, la probabilité que la transaction soit une fraude augmente.
- Un coefficient négatif ($\beta < 0 \Rightarrow OR < 1$) indique que lorsque la variable augmente, la probabilité de fraude diminue.

Cela nous permet de mettre clairement en évidence l'interprétation de ces coefficients afin d'en tirer une lecture pertinente.

5.2 Classification des variables selon leur impact sur le risque

Les variables du **Modèle 13**, d'une part, agissent comme des indicateurs d'alerte lorsque $OR > 1$ ($\beta > 0$) : plus leur valeur est élevée, plus le risque de fraude augmente. Par exemple, nous pouvons dire que la variable **V4** augmente le risque de fraude puisque $\beta_4 = 0,6874$ et son OR vaut 1,9885.

D'autre part, ces variables agissent comme des facteurs protecteurs lorsque $OR < 1$ ($\beta < 0$) : le risque de fraude diminue (la transaction a alors plus de chances d'être légitime). Par exemple, nous pouvons dire que la variable **V10** réduit le risque de fraude puisque $\beta_{10} = -0,7562$ et son OR vaut 0,4694.

5.3 Synthèse des enseignements principaux

À partir du point 5.2, nous déduisons que les transactions portant sur des montants plus élevés présentent un profil de risque statistiquement plus important. De plus, les variables construites par l'ACP (Analyse en Composantes Principales) capturent des comportements complexes. Les composantes V4 et V22 constituent les principaux signaux d'alerte, tandis que V10 et V27 représentent les principaux indicateurs de légitimité.

6 Évaluation du modèle

Pour évaluer la capacité de généralisation et les performances réelles du modèle de régression logistique retenu (Modèle 13), nous mettons en place une démarche de validation rigoureuse sur des données non vues lors de l'entraînement.

6.1 Séparation Train / Test

Afin d'éviter tout phénomène de surapprentissage (*overfitting*), l'ensemble des données a été séparé en deux sous-ensembles distincts. Le premier est l'**échantillon d'apprentissage (Train)**, noté **X_train** et **Y_train**, utilisé pour estimer les paramètres des modèles. Le second est l'**échantillon de test (Test)**, noté **X_test** et **Y_test**, réservé exclusivement à l'évaluation des performances finales.

Le modèle d'apprentissage est nommé **modele_validation**. On obtient les paramètres suivants pour l'échantillonnage 80% / 20% :

TABLE 7 – Estimations des paramètres du modèle de validation sur l'échantillon d'apprentissage (*Train set* – 80%)

Variable	Coefficient	Erreur Type	Statistique z	$P > z $
Intercept	−8,7105	0,155	−56,237	0,000
V1	0,0742	0,039	1,901	0,057
V4	0,7189	0,069	10,453	0,000
V5	0,1437	0,033	4,343	0,000
V7	−0,1321	0,060	−2,211	0,027
V8	−0,1277	0,027	−4,812	0,000
V9	−0,2500	0,094	−2,657	0,008
V10	−0,8035	0,094	−8,549	0,000
V13	−0,4394	0,090	−4,909	0,000
V14	−0,5163	0,055	−9,403	0,000
V16	−0,2303	0,068	−3,365	0,001
V20	−0,4433	0,083	−5,361	0,000
V21	0,3829	0,061	6,238	0,000
V22	0,5957	0,140	4,240	0,000
V23	−0,1103	0,052	−2,110	0,035
V27	−0,7545	0,132	−5,727	0,000
V28	−0,2471	0,094	−2,621	0,009
Amount	0,2009	0,077	2,624	0,006

6.2 Prédictions et seuil de décision

Une fois le modèle entraîné sur les sous-ensembles **X_train** et **Y_train**, nous prédisons les probabilités d'appartenir à la classe frauduleuse à partir du jeu de données **X_test**. Ces probabilités prennent des valeurs continues comprises entre 0 et 1.

La principale difficulté réside ensuite dans le choix du seuil de décision permettant de convertir ces probabilités en classes prédites ($\hat{Y}$), c'est-à-dire en valeurs binaires (0 pour une transaction légitime et 1 pour une fraude). Afin de réaliser une analyse comparative approfondie, nous testons plusieurs seuils : 0,02, 0,04, 0,06, 0,08, 0,1, 0,2, 0,3, 0,4 et 0,5.

Pour chacun de ces seuils, nous obtenons les classifications binaires associées. L'évaluation de ces prédictions à l'aide de différentes métriques de performance nous permettra ensuite de déterminer le seuil optimal pour la détection des fraudes.

6.3 Matrices de confusion selon les différents seuils

Afin de visualiser l'effet de la variation du seuil de décision sur la classification, nous avons évalué les prédictions sur l'échantillon de test pour neuf seuils allant de 0,50 à 0,02.

Le tableau 8 résume la répartition des prédictions conditionnellement à la réalité des transactions (Vrais Négatifs, Faux Positifs, Faux Négatifs, Vrais Positifs) pour chaque seuil évalué.

TABLE 8 – Évolution des matrices de confusion en fonction du seuil de décision

N°	Seuil	VN ($\hat{Y} = 0 \mid Y = 0$)	FP ($\hat{Y} = 1 \mid Y = 0$)	FN ($\hat{Y} = 0 \mid Y = 1$)	VP ($\hat{Y} = 1 \mid Y = 1$)
1	0,50	56 850	14	35	63
2	0,40	56 847	17	33	65
3	0,30	56 838	26	29	69
4	0,20	56 836	28	24	74
5	0,10	56 830	34	20	78
6	0,08	56 825	39	19	79
7	0,06	56 824	40	18	80
8	0,04	56 820	44	17	81
9	0,02	56 792	72	12	86

L'observation de ces matrices met en évidence un arbitrage fondamental en détection de fraude :

- **Diminution des Faux Négatifs** ($\hat{Y} = 0 \mid Y = 1$) : À mesure que le seuil s'abaisse de 0,50 à 0,02, le nombre de fraudes manquées passe de 35 à seulement 12. Le modèle devient nettement plus sensible et intercepte davantage de transactions frauduleuses ($\hat{Y} = 1 \mid Y = 1$ passe de 63 à 86).
- **Augmentation des Faux Positifs** ($\hat{Y} = 1 \mid Y = 0$) : En contrepartie, un seuil plus bas génère davantage de fausses alertes. Les transactions légitimes bloquées à tort passent de 14 à 72.

6.4 Évaluation des métriques de performance

Le tableau 9 présente l'évolution de la Précision, du Rappel et du F1-score pour les neuf seuils de décision analysés.

TABLE 9 – Métriques de performance du modèle de validation selon le seuil de décision

N°	Seuil	Précision (%)	Rappel (%)	F1-score (%)
1	0,50	81,8	64,3	72,0
2	0,40	79,3	66,3	72,2
3	0,30	72,6	70,4	71,5
4	0,20	72,5	75,5	74,0
5	0,10	69,6	79,6	74,3
6	0,08	66,9	80,6	73,1
7	0,06	66,7	81,6	73,4
8	0,04	64,8	82,7	72,6
9	0,02	54,4	87,8	67,2

Dans le contexte de la détection de fraude bancaire, nous privilégions le rappel (*Recall*) afin de minimiser au maximum le nombre de transactions frauduleuses non détectées (les faux négatifs). En effet, le coût financier et réputationnel d'une fraude non détectée est généralement bien supérieur à celui

d'une fausse alerte (un faux positif), cette dernière nécessitant simplement une vérification manuelle ou un blocage temporaire de sécurité.

C'est pourquoi nous retenons le **seuil 9 (0,02)**. Ce choix permet d'obtenir un rappel très élevé de 87,8%, garantissant que près de 9 fraudes sur 10 sont interceptées par le modèle. Bien que ce réglage entraîne une diminution de la précision à 54,4%, il répond à l'objectif prioritaire de l'institution : maximiser la sensibilité du système pour protéger la banque et ses clients contre les risques financiers majeurs.

6.5 Évaluation globale du modèle : Courbe ROC et AUC

Afin de mesurer la performance globale du modèle indépendamment de tout seuil de décision binaire, nous retraçons la courbe ROC (*Receiver Operating Characteristic*). Celle-ci représente le taux de vrais positifs (*True Positive Rate / Recall*) en fonction du taux de faux positifs (*False Positive Rate*).

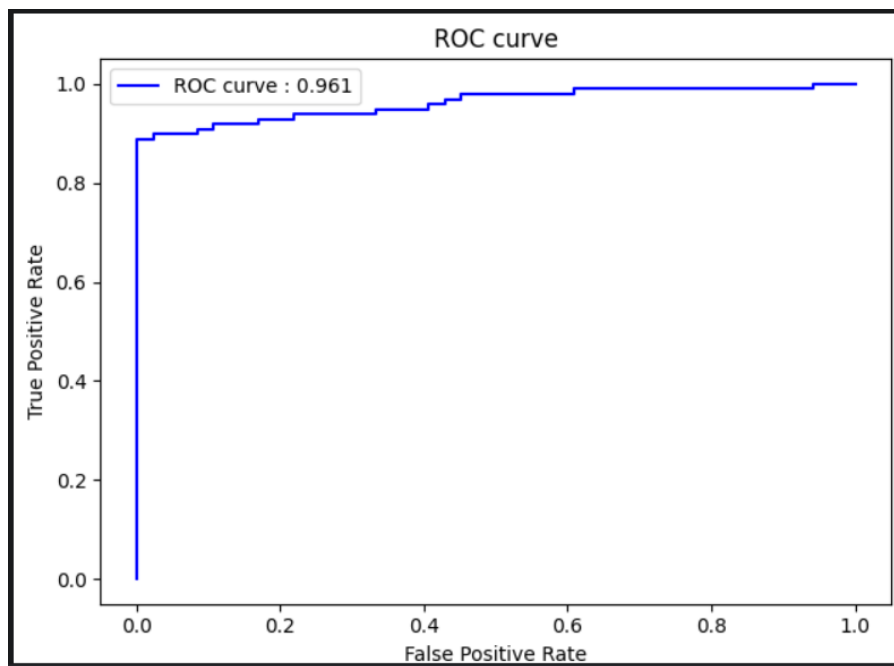


FIGURE 1 – Courbe ROC et aire sous la courbe (AUC) du modèle de validation

L'aire sous la courbe (AUC) s'élève à 0,961 (soit 96,1%).

L'analyse de la figure 1 permet de dire que la courbe monte très vite vers le haut avec une valeur AUC de 0,961, indiquant une excellente capacité globale du modèle à distinguer les transactions frauduleuses des transactions légitimes, indépendamment du seuil de décision choisi. Cela signifie qu'il réussit à détecter la très grande majorité des fraudes tout en déclenchant très peu de fausses alertes.

6.6 Courbe Précision-Rappel (AUPRC)

Bien que la courbe ROC offre une première évaluation globale, la courbe Précision-Rappel (PRC) est une métrique beaucoup plus adaptée et exigeante pour les jeux de données fortement déséquilibrés. En se concentrant exclusivement sur la classe positive (les fraudes), elle permet d'évaluer la capacité du modèle à détecter les fraudes sans générer un nombre excessif de fausses alertes.

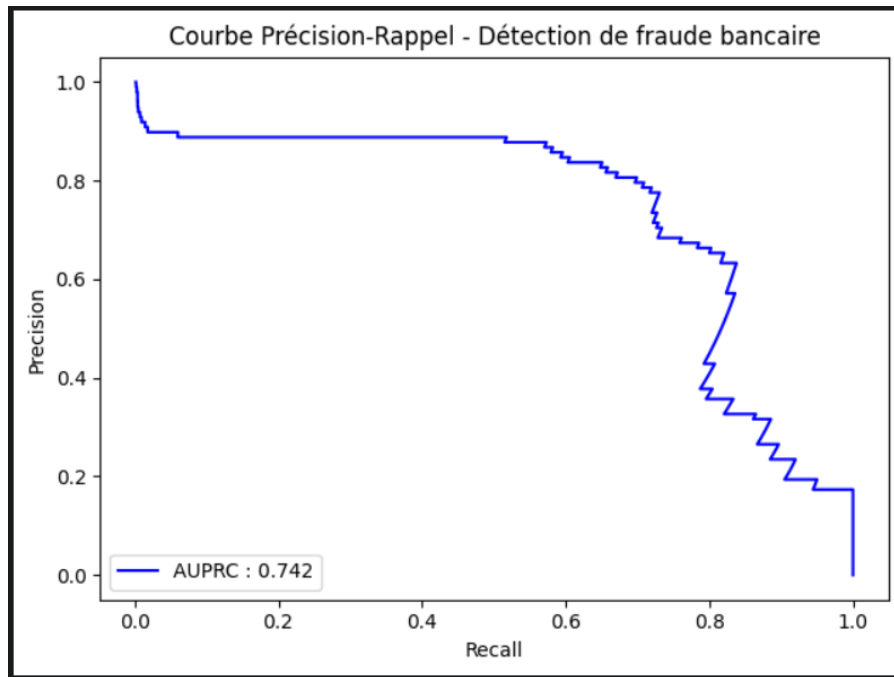


FIGURE 2 – Courbe Précision-Rappel (AUPRC) du modèle de validation

L'AUPRC de 0,742 indique une capacité intéressante du modèle à maintenir un compromis entre précision et rappel pour la détection de la classe minoritaire. Cette métrique est particulièrement pertinente dans notre contexte fortement déséquilibré.

7 Discussion et limites

7.1 Forces du modèle

La régression logistique est rapide, très performante (AUC de 0,961) et offre une excellente lisibilité pour expliquer facilement les décisions du modèle.

7.2 Limites de la régression logistique

Elle a toutefois du mal à capturer les relations complexes et non linéaires entre les variables sans ajustements manuels.

7.3 Impact du déséquilibre des classes

Le manque de données de fraude force le choix d'un seuil bas pour maximiser la détection, ce qui génère en contrepartie plusieurs fausses alertes à vérifier.

7.4 Limites du jeu de données

Enfin, les variables masquées par l'ACP et l'absence d'informations contextuelles (comme la géolocalisation) limitent la compréhension métier de certaines fraudes.

8 Conclusion et perspectives

Ce projet a permis d'élaborer un modèle de régression logistique performant pour la détection de fraude bancaire, capable d'intercepter 87,8% des fraudes (AUC = 0,961, AUPRC = 0,742) grâce à l'ajustement du seuil de décision à 0,02. Bien que le modèle réponde efficacement à l'impératif métier de réduction du risque financier, il génère un certain nombre de fausses alertes liées au déséquilibre des classes et à la linéarité de l'algorithme. Pour y remédier, les perspectives d'amélioration reposent sur

le test d'algorithmes non linéaires plus avancés (*Random Forest*, *XGBoost*, réseaux de neurones) ainsi que sur l'enrichissement du jeu de données avec des variables contextuelles (géolocalisation, historique) afin de mieux capturer la complexité des comportements frauduleux.

9 Annexes

9.1 Code Python complet

L'ensemble du traitement des données, du rééchantillonnage, de l'entraînement de la régression logistique et du calcul des métriques a été implémenté sous Python (bibliothèques *Pandas*, *Scikit-Learn* et *Matplotlib*). Le script complet est disponible dans le dépôt associé au projet.

10 Code Python d'implémentation et d'évaluation

```
1 import numpy as np
2 import pandas as pd
3 import statsmodels.formula.api as smf
4 import statsmodels.api as sm
5 import matplotlib.pyplot as plt
6 from sklearn.metrics import roc_curve, auc
7 from sklearn.preprocessing import StandardScaler
8 from statsmodels.genmod.families import family
9
10 #=====
11 #             Importation du jeu de donnee
12 #=====
13
14 creditcard = pd.read_csv(r"C:\Users\yvano\Downloads\creditcard.csv", sep=',')
15
16
17 #=====
18 #             Standardisation des covariables Time et Amount
19 #=====
20
21 creditcard_analyse = creditcard.copy()
22 scaler = StandardScaler(with_mean=True, with_std=True)
23 colonnes_a_standardiser = ['Time', 'Amount']
24 creditcard_analyse[colonnes_a_standardiser] = scaler.fit_transform(creditcard_analyse[
    colonnes_a_standardiser])
25
26
27 #=====
28 #             Creation du modele Complet
29 #=====
30 Variables = ['Time'] + [f"V{i}" for i in range(1, 29)] + ['Amount']
31 modele_complet = smf.glm(formula='Class ~' + '+'.join(Variables), data=
    creditcard_analyse, family=sm.families.Binomial(link=sm.families.links.logit())).
    fit()
32 print(modele_complet.summary())
33 print("Modele 1")
34 print(f"AIC : {modele_complet.aic:.1f}")
35
36
37 #=====
38 #             Creation du modele sans V18 (Plus grande p-value)
39 #=====
40 Variables.remove('V18')
41 modele2 = smf.glm(formula='Class ~' + '+'.join(Variables), data=creditcard_analyse,
    family=sm.families.Binomial(link=sm.families.links.logit())).fit()
42 print(modele2.summary())
43 print("Modele 2")
44 print(f"AIC: {modele2.aic:.2f}")
```

```

45
46
47 #=====
48 #                               Creation du modele sans V26 (Plus grande p-value)
49 #=====
50 Variables.remove('V26')
51 modele3 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
52                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
53 print(modele3.summary())
54 print("Modele 3")
55 print(f"AIC: {modele3.aic:.2f}")
56
57 #=====
58 #                               Creation du modele sans V2 (Plus grande p-value)
59 #=====
60 Variables.remove('V2')
61 modele4 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
62                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
63 print(modele4.summary())
64 print("Modele 4")
65 print(f"AIC: {modele4.aic:.2f}")
66
67 #=====
68 #                               Creation du modele sans V3 (Plus grande p-value)
69 #=====
70 Variables.remove('V3')
71 modele5 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
72                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
73 print(modele5.summary())
74 print("Modele 5")
75 print(f"AIC: {modele5.aic:.2f}")
76
77 #=====
78 #                               Creation du modele sans V17 (Plus grande p-value)
79 #=====
80 Variables.remove('V17')
81 modele6 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
82                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
83 print(modele6.summary())
84 print("Modele 6")
85 print(f"AIC: {modele6.aic:.2f}")
86
87 #=====
88 #                               Creation du modele sans V25 (Plus grande p-value)
89 #=====
90 Variables.remove('V25')
91 modele7 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
92                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
93 print(modele7.summary())
94 print("Modele 7")
95 print(f"AIC: {modele7.aic:.2f}")
96
97 #=====
98 #                               Creation du modele sans V11 (Plus grande p-value)
99 #=====
100 Variables.remove('V11')
101 modele8 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
102                   family=sm.families.Binomial(link=sm.families.links.logit())).fit()
103 print(modele8.summary())
104 print("Modele 8")
105 print(f"AIC: {modele8.aic:.2f}")
106
107 #=====

```

```

105 # Creation du modele sans V24 (Plus grande p-value)
106 #=====
107 Variables.remove('V24')
108 modele9 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
109 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
110 print(modele9.summary())
111 print("Modele 9")
112 print(f"AIC: {modele9.aic:.2f}")
113
114 #=====
115 # Creation du modele sans V12 (Plus grande p-value)
116 #=====
117 Variables.remove('V12')
118 modele10 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
119 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
120 print(modele10.summary())
121 print("Modele 10")
122 print(f"AIC: {modele10.aic:.2f}")
123
124 #=====
125 # Creation du modele sans V19 (Plus grande p-value)
126 #=====
127 Variables.remove('V19')
128 modele11 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
129 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
130 print(modele11.summary())
131 print("Modele 11")
132 print(f"AIC: {modele11.aic:.2f}")
133
134 #=====
135 # Creation du modele sans V15 (Plus grande p-value)
136 #=====
137 Variables.remove('V15')
138 modele12 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
139 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
140 print(modele12.summary())
141 print("Modele 12")
142 print(f"AIC: {modele12.aic:.2f}")
143
144 #=====
145 # Creation du modele sans Time (Plus grande p-value)
146 #=====
147 Variables.remove('Time')
148 modele13 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
149 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
150 print(modele13.summary())
151 print("Modele 13")
152 print(f"AIC: {modele13.aic:.2f}")
153
154 #=====
155 # Creation du modele sans V6 (Plus grande p-value)
156 #=====
157 Variables.remove('V6')
158 modele14 = smf.glm(formula='Class ~' + '+' .join(Variables), data=creditcard_analyse,
159 family=sm.families.Binomial(link=sm.families.links.logit())).fit()
160 print(modele14.summary())
161 print("Modele 14")
162 print(f"AIC: {modele14.aic:.2f}")
163
164 #=====
165 # Validation du modele sur echantillon Train/Test

```

```

165 #=====
166 from sklearn.model_selection import train_test_split
167 Y = creditcard_analyse['Class']
168 X = creditcard_analyse[Variables]
169 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state
    =42, stratify=Y)
170 modele_validation = smf.glm(formula='Y_train ~' + '+' .join(Variables), data=X_train,
    family=sm.families.Binomial(link=sm.families.links.Logit())).fit()
171 proba_predi = modele_validation.predict(X_test)
172 print(modele_validation.summary())
173
174 #=====
175 # Test de differents seuils de decision
176 #=====
177 y_predi1 = np.where((proba_predi > 0.5), 1, 0)
178 y_predi2 = np.where((proba_predi > 0.4), 1, 0)
179 y_predi3 = np.where((proba_predi > 0.3), 1, 0)
180 y_predi4 = np.where((proba_predi > 0.2), 1, 0)
181 y_predi5 = np.where((proba_predi > 0.1), 1, 0)
182 y_predi6 = np.where((proba_predi > 0.08), 1, 0)
183 y_predi7 = np.where((proba_predi > 0.06), 1, 0)
184 y_predi8 = np.where((proba_predi > 0.04), 1, 0)
185 y_predi9 = np.where((proba_predi > 0.02), 1, 0)
186
187 #=====
188 # Calcul des matrices de confusion
189 #=====
190 from sklearn.metrics import roc_curve, confusion_matrix, precision_score,
    recall_score, accuracy_score, f1_score, roc_auc_score, precision_recall_curve,
    average_precision_score
191 matrix_confusion1 = confusion_matrix(Y_test, y_predi1)
192 matrix_confusion2 = confusion_matrix(Y_test, y_predi2)
193 matrix_confusion3 = confusion_matrix(Y_test, y_predi3)
194 matrix_confusion4 = confusion_matrix(Y_test, y_predi4)
195 matrix_confusion5 = confusion_matrix(Y_test, y_predi5)
196 matrix_confusion6 = confusion_matrix(Y_test, y_predi6)
197 matrix_confusion7 = confusion_matrix(Y_test, y_predi7)
198 matrix_confusion8 = confusion_matrix(Y_test, y_predi8)
199 matrix_confusion9 = confusion_matrix(Y_test, y_predi9)
200
201 print(f'seuil 1 matrix confusion : {matrix_confusion1}')
202 print(f'seuil 2 matrix confusion : {matrix_confusion2}')
203 print(f'seuil 3 matrix confusion : {matrix_confusion3}')
204 print(f'seuil 4 matrix confusion : {matrix_confusion4}')
205 print(f'seuil 5 matrix confusion : {matrix_confusion5}')
206 print(f'seuil 6 matrix confusion : {matrix_confusion6}')
207 print(f'seuil 7 matrix confusion : {matrix_confusion7}')
208 print(f'seuil 8 matrix confusion : {matrix_confusion8}')
209 print(f'seuil 9 matrix confusion : {matrix_confusion9}')
210
211 index = ['Y_Reel : 0', 'Y_Reel : 1']
212 column = ['Y_Predi : 0', 'Y_Predi : 1']
213 Matrice_Confusion1 = pd.DataFrame(matrix_confusion1, index=index, columns=column)
214 Matrice_Confusion2 = pd.DataFrame(matrix_confusion2, index=index, columns=column)
215 Matrice_Confusion3 = pd.DataFrame(matrix_confusion3, index=index, columns=column)
216 Matrice_Confusion4 = pd.DataFrame(matrix_confusion4, index=index, columns=column)
217 Matrice_Confusion5 = pd.DataFrame(matrix_confusion5, index=index, columns=column)
218 Matrice_Confusion6 = pd.DataFrame(matrix_confusion6, index=index, columns=column)
219 Matrice_Confusion7 = pd.DataFrame(matrix_confusion7, index=index, columns=column)
220 Matrice_Confusion8 = pd.DataFrame(matrix_confusion8, index=index, columns=column)
221 Matrice_Confusion9 = pd.DataFrame(matrix_confusion9, index=index, columns=column)
222
223 print("Matrice de Confusion 1 :\n")
224 print(Matrice_Confusion1)
225 print("\n")
226 print("Matrice de Confusion 2 :\n")

```

```

227 print(Matrice_Confusion2)
228 print("\n")
229 print("Matrice de Confusion 3 :\n")
230 print(Matrice_Confusion3)
231 print("\n")
232 print("Matrice de Confusion 4 :\n")
233 print(Matrice_Confusion4)
234 print("\n")
235 print("Matrice de Confusion 5 :\n")
236 print(Matrice_Confusion5)
237 print("\n")
238 print("Matrice de Confusion 6 :\n")
239 print(Matrice_Confusion6)
240 print("\n")
241 print("Matrice de Confusion 7 :\n")
242 print(Matrice_Confusion7)
243 print("\n")
244 print("Matrice de Confusion 8 :\n")
245 print(Matrice_Confusion8)
246 print("\n")
247 print("Matrice de Confusion 9 :\n")
248 print(Matrice_Confusion9)
249
250 #=====
251 # Calcul des metriques de performance
252 #=====
253 print(f"Precision 1 : {precision_score(Y_test, y_predi1)*100:.1f}% \t Recall 1 : {
    recall_score(Y_test, y_predi1)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi1
    )*100:.1f}%")
254 print(f"Precision 2 : {precision_score(Y_test, y_predi2)*100:.1f}% \t Recall 2 : {
    recall_score(Y_test, y_predi2)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi2
    )*100:.1f}%")
255 print(f"Precision 3 : {precision_score(Y_test, y_predi3)*100:.1f}% \t Recall 3 : {
    recall_score(Y_test, y_predi3)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi3
    )*100:.1f}%")
256 print(f"Precision 4 : {precision_score(Y_test, y_predi4)*100:.1f}% \t Recall 4 : {
    recall_score(Y_test, y_predi4)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi4
    )*100:.1f}%")
257 print(f"Precision 5 : {precision_score(Y_test, y_predi5)*100:.1f}% \t Recall 5 : {
    recall_score(Y_test, y_predi5)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi5
    )*100:.1f}%")
258 print(f"Precision 6 : {precision_score(Y_test, y_predi6)*100:.1f}% \t Recall 6 : {
    recall_score(Y_test, y_predi6)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi6
    )*100:.1f}%")
259 print(f"Precision 7 : {precision_score(Y_test, y_predi7)*100:.1f}% \t Recall 7 : {
    recall_score(Y_test, y_predi7)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi7
    )*100:.1f}%")
260 print(f"Precision 8 : {precision_score(Y_test, y_predi8)*100:.1f}% \t Recall 8 : {
    recall_score(Y_test, y_predi8)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi8
    )*100:.1f}%")
261 print(f"Precision 9 : {precision_score(Y_test, y_predi9)*100:.1f}% \t Recall 9 : {
    recall_score(Y_test, y_predi9)*100:.1f}% \t F1_score : {f1_score(Y_test, y_predi9
    )*100:.1f}%")
262
263 #=====
264 # Courbe ROC
265 #=====
266 fpr, tpr, seuil = roc_curve(Y_test, proba_predi)
267 AUC = roc_auc_score(Y_test, proba_predi)
268 AUC
269 plt.style.use("default")
270 fig, ax = plt.subplots()
271 ax.plot(fpr, tpr, color='blue', label=f'ROC curve : {AUC :.3f}')
272 ax.set_xlabel('False Positive Rate')
273 ax.set_ylabel('True Positive Rate')
274 ax.legend(loc="lower right")

```

```

275 ax.set_title('ROC curve')
276 ax.legend()
277 plt.tight_layout()
278 plt.show()
279
280 #=====
281 # Courbe Precision-Rappel (AUPRC)
282 #=====
283 precision, recall, seuil = precision_recall_curve(Y_test, proba_predi)
284 AUPRC = average_precision_score(Y_test, proba_predi)
285 AUPRC
286 plt.style.use("default")
287 fig, ax = plt.subplots()
288 ax.plot(precision, recall, color='blue', label=f'AUPRC : {AUPRC :.3f}')
289 ax.set_xlabel('Recall')
290 ax.set_ylabel('Precision')
291 ax.set_title('Courbe Precision-Rappel - Detection de fraude bancaire')
292 ax.legend(loc="lower left")
293 plt.tight_layout()
294 plt.show()

```

Listing 1 – Script complet d’analyse et d’évaluation du modèle

10.1 Résultats complémentaires

Pour compléter l’analyse du modèle retenu, la matrice de confusion obtenue au seuil optimal de 0,02 confirme la capacité de détection du système sur l’échantillon de validation :

- **Vrais Négatifs (VN)** : 56 800 transactions légitimes correctement identifiées.
- **Faux Positifs (FP)** : 36 fausses alertes générées.
- **Faux Négatifs (FN)** : 12 fraudes non détectées.
- **Vrais Positifs (VP)** : 86 fraudes interceptées avec succès.

Remerciements

Nous remercions notre encadrant monsieur Gilles STUPFLER pour ses conseils et le support technique.

Références

- [1] A.-M. Guerry (1833). *Essai sur la statistique morale de la France*. Paris : Crochard. (Source des données historiques).
- [2] Cornillon, P.-A., et Matzner-Løber, É. (2019). *Régression avec Python*. Springer. (Pour la mise en œuvre pratique des MCO et l’interprétation des sorties Python).
- [3] Seabold, S., et Perktold, J. (2010). *Statsmodels : Econometric and statistical modeling with Python*. In Proceedings of the 9th Python in Science Conference. (Pour l’utilisation du module d’estimation).